

INTERNSHIP REPORT ON
COLLEGE ADMISSION MANAGEMENT SYSTEM USING
PYTHON, TKINTER AND MYSQL

Submitted by: Kanika

Roll no: 241119061012



Submitted to :- Dr. Mamta Rani

(Associate Professor)

Department of BCA

Kishan Lal Public College, Rewari

Internship Duration: 3rd July – 30th July 2026

Date of Submission: 12-08-2026

CERTIFICATE

I, **Kanika**, hereby certify that this Internship Report entitled “**College Admission Management System using Python, Tkinter and MySQL**” is an original record of the work carried out by me under the guidance of **Mr. Kamal Kumar**, Internship Mentor, **Zillionsofttech**, during the internship period from **3rd July 2026 to 30th July 2026** for a duration of **four weeks**.

I further certify that this work was completed under the supervision of **Dr. Mamta Rani**, Associate Professor, Department of BCA, **Kishan Lal Public College (Rewari)**, and is respectfully submitted in partial fulfillment of the internship requirement prescribed to be undertaken after the completion of the **4th Semester** of the **Bachelor of Computer Applications (BCA)** programme in the Department of BCA, Kishan Lal Public College (Rewari).

Dated: _____

Signature of Student: _____

Name: Kanika

Roll No.:241119061038

Countersigned by:

Internship Supervisor

Department of BCA, Kishan Lal Public College, Rewari

ACKNOWLEDGEMENT

I want to express my sincere gratitude to my project guide for providing valuable guidance, continuous support, and encouragement throughout the development of this project.

I am thankful to the Department of Computer Science and all the faculty members for providing the necessary resources and knowledge required for the successful completion of this project.

I also extend my heartfelt thanks to my family and friends for their constant motivation, encouragement, and support during the project work.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of the **College Admission Management System** project.

Submitted By

Kanika Verma

Bachelor of Computer Applications (BCA)

Academic Session: **2026–2027**

TABLE OF CONTENTS

S No.	Table of Content	Page No.
01	Certificate	1
02	Acknowledgement	2
03	Abstract	3
04	About the Organization	4
05	Introduction	6
06	Objectives of the Internship	7
07	Technologies Learned During the Internship	8
08	Project Overview	9
09	Features of the Project	10
10	Project Structure (MVC Architecture)	12
11	Database Design	15
12	Database Tables	16
13	Important Code Snippets	18
14	Project Implementations	21
15	System Testing	29
16	Advantages of the System	31
17	Limitations of the System	32
18	Learning Outcomes	33
19	Challenges Faced and Solutions	34
20	Conclusion / Summary	36

ABSTRACT

The **College Admission Management System** is a desktop-based application developed using **Python**, **Tkinter**, and **MySQL**. The system is designed to automate and simplify the student admission process by replacing manual record management with a digital solution.

The application enables users to manage student admissions efficiently by storing student details, course information, admission records, and fee details in a centralized MySQL database. It provides functionalities such as adding new student records, searching students, updating information, deleting records, and viewing all admissions through an easy-to-use graphical user interface.

The project follows the **Model-View-Controller (MVC)** architecture along with the **Data Access Object (DAO)** design pattern, making the application modular, maintainable, and scalable. Using MySQL ensures secure and reliable data storage, while the Tkinter-based graphical interface allows users to interact with the system without requiring technical knowledge.

This project demonstrates the practical implementation of database connectivity, GUI development, object-oriented programming, and software engineering principles to build an efficient college admission management solution.

ABOUT THE ORGANIZATION

Zillionsofttech is a technical training and software development organization located at Model Town, Rewari, Haryana. Established in 2020, the organization operates in software development, desktop and web application development, data analytics, and IT skills training. The organization conducts industry-oriented internship programmes in which students work on live, real-world projects under the mentorship of experienced professionals.

Vision and Mission

The vision of the organization is to bridge the gap between academic learning and industry requirements by producing skilled, employable, and industry-ready professionals. Its mission is to impart practical, hands-on training on widely used technologies such as Python, MySQL, Java, and data analytics tools,

Areas of Work

- Application development using Python (Desktop GUI applications using Tkinter, automation scripts, and data processing)
- Database design and management using MySQL
- Data Analytics using Python (Pandas, NumPy), Excel, and Power BI
- Web Development and Digital Marketing
- Corporate training, student internship, and placement assistance programmes

Organizational Structure

The organization functions supported by project managers, senior developers, trainers, and support staff. Interns are attached to a project team and work under the direct guidance of a senior developer/trainer who acts as the Internship Mentor. Weekly review meetings are conducted to monitor the progress of the interns and the project deliverables.

Work Culture and Facilities

The work culture of the organization is collaborative and learning-oriented. Daily task allotment, regular code reviews, task tracking, and proper documentation are integral parts of the workflow. The organization provides a fully equipped computer lab with the required software (Python, MySQL Server, MySQL Workbench, VS Code) and internet facility, which helped me understand the professional practices followed in the software industry.

Why This Organization Was Chosen

I selected this organization for my internship because it offers project-based training on Python and MySQL, which are directly related to my area of study and career interest. Being a local organization, it was also convenient in terms of accessibility, and it fulfils the criteria of an Internship Providing Organization (IPO)

as laid down in the Internship Guidelines of Indira Gandhi University, Meerpur.

INTRODUCTION

The **Summer Internship** was organized to provide practical exposure to software development and enhance programming skills through real-world application development. During this internship, I learned several important technologies, including **Python**, **MySQL**, **Tkinter**, **Git**, and **GitHub**. These technologies helped me understand the complete process of designing, developing, managing, and maintaining a software application.

Python was used as the primary programming language because of its simplicity, readability, and extensive library support. Tkinter was used to develop an interactive Graphical User Interface (GUI), allowing users to interact with the application easily. MySQL was used as the database management system to store, retrieve, update, and manage student information efficiently. Git and GitHub were used for version control, enabling proper management of the project's source code and tracking changes throughout the development process.

To apply the knowledge gained during the internship, I developed a **College Admission Management System**. The project is a desktop-based application designed to simplify and automate the college admission process. It allows users to maintain student records, manage course details, monitor fee information, and perform operations such as adding, searching, updating, deleting, and viewing student records through a user-friendly interface.

The project follows the **Model-View-Controller (MVC)** architecture and implements the **Data Access Object (DAO)** design pattern, which separates the user interface, business logic, and database operations into different modules. This approach improves the maintainability, scalability, and organization of the application.

Overall, the Summer Internship provided valuable practical experience in software development, database management, graphical user interface design, and version control. It strengthened my technical knowledge and helped me understand how different technologies work together to develop a complete desktop application.

OBJECTIVES OF INTERNSHIP

The main objective of this Summer Internship was to gain practical knowledge of software development and improve my programming skills. During the internship, I learned how different technologies work together to develop a complete desktop application.

The objectives of this internship are:

- To learn the basics of **Python** programming.
- To understand how to create a **Graphical User Interface (GUI)** using **Tkinter**.
- To learn how to store and manage data using the **MySQL** database.
- To understand how to connect Python applications with a MySQL database.
- To learn the use of **Git** and **GitHub** for version control and project management.
- To understand the **MVC (Model-View-Controller)** architecture and **DAO (Data Access Object)** design pattern.
- To develop a real-world desktop application named **College Admission Management System**.
- To improve problem-solving, debugging, and logical thinking skills.
- To gain confidence in developing software applications independently.

TECHNOLOGIES LEARNED DURING THE INTERNSHIP

During the Summer Internship, I learned several technologies that helped me develop the **College Admission Management System**. Each technology played an important role in the development of the project.

1. Python

Python is a simple and easy-to-learn programming language. It was used to develop the main logic of the application. Python made it easy to write clean and organized code and helped in building the complete desktop application.

2. Tkinter

Tkinter is Python's built-in library for creating Graphical User Interfaces (GUI). It was used to design the windows, buttons, labels, text fields, and other components of the application. It helped in creating a simple and user-friendly interface.

3. MySQL

MySQL is a Relational Database Management System (RDBMS). It was used to store student details, course information, admission records, and fee details. MySQL allows data to be stored securely and retrieved whenever needed.

4. Git

Git is a version control system that helps developers keep track of changes made to a project. During the internship, I learned how to save different versions of my project, manage code changes, and work in an organized manner.

5. GitHub

GitHub is an online platform used to store and manage Git repositories. It was used to upload the project, maintain backups, and keep the project code available for future use. It also helps in sharing projects and collaborating with other developers.

PROJECT OVERVIEW

The College Admission Management System is a desktop-based application developed during the Summer Internship using Python, Tkinter, and MySQL. The purpose of this project is to make the college admission process easier, faster, and more organized.

In many colleges, student admission records are maintained manually, which takes more time and can lead to errors. This system helps in storing and managing student information digitally. It allows the user to add new student records, search student details, update existing information, delete records, and view all admission records through a simple graphical interface.

The application is connected to a MySQL database, where all student, course, and fee information is stored securely. The Tkinter GUI makes the system easy to use, even for users who do not have advanced technical knowledge.

The project is developed using the MVC (Model-View-Controller) architecture and DAO (Data Access Object) design pattern. These design patterns help in separating the user interface, business logic, and database operations, making the application more organized and easier to maintain.

Main Features of the System

- Add new student admission records
- Search student details by Student ID
- Update student information
- Delete student records
- View all admitted students
- Manage course details
- Maintain fee records (paid and pending fees)
- User-friendly graphical interface

This project helped me understand how real-world desktop applications are developed and how Python can be used with databases to create useful management systems.

FEATURES OF THE PROJECT

The **College Admission Management System** is designed to make the admission process simple, fast, and well-organized. The application provides different features that help users manage student admission records efficiently. The main features of the system are as follows:

1. Student Registration

The system allows users to enter and save the details of new students, such as their name, father's name, gender, date of birth, mobile number, email, address, and selected course.

2. Course Management

Users can select a course for each student during the admission process. The course information is stored in the database and linked with the student's admission record.

3. Fee Management

The system stores both **paid fees** and **pending fees** for every student. This helps in tracking the fee status easily.

4. Search Student

Users can quickly search for a student's record using the **Student ID**. This saves time and makes it easy to find the required information.

5. Update Student Details

If any student information changes, the user can update the existing record without creating a new admission entry.

6. Delete Student Record

The system allows users to remove student records that are no longer required or were entered incorrectly.

7. View All Student Records

All admission records can be displayed in a table format, making it easy to view and manage student information.

8. Secure Data Storage

All student records are stored in a **MySQL database**, which helps in maintaining data safely and reducing the chances of data loss.

9. User-Friendly Interface

The application has a simple, easy-to-use graphical interface built with Tkinter, allowing users to perform various tasks without difficulty.

PROJECT STRUCTURE (MVC ARCHITECTURE)

The **College Admission Management System** is developed using the **MVC (Model-View-Controller)** architecture. MVC is a software design pattern that separates the application into different parts. This makes the code more organized, easier to understand, and easier to maintain.

The project also follows the **DAO (Data Access Object)** design pattern for handling database operations. The DAO layer is responsible for communicating with the MySQL database, while the MVC architecture manages the overall structure of the application.

The project is organized into the following folders:

Project Directory Structure

```
CollegeAdmissionManagementSystem/
├── main.py                    # Application entry point
├── README.md                  # Project documentation
├── config/                    # Configuration files
│   └── db_connection.py      # MySQL database connection
├── models/                    # Model classes
│   └── student.py            # Student model class
├── dao/                        # Data Access Object layer
│   └── admission_dao.py      # Database CRUD operations
├── controllers/               # Controller layer
│   └── admission_controller.py # Admission Controller
├── services/                  # Business logic layer
│   └── admission_service.py  # Admission Service
├── views/                      # Tkinter GUI screens
│   ├── admission_form.py     # Student Admission Form
│   └── student_table.py      # Student Records Table
```

Description of Project Files

1. main.py

- This is the entry point of the application.
- It initializes the project and launches the main window.

- It connects all the project components.

2. config Folder

db_connection.py

- Stores the MySQL database connection code.
- Establishes a connection between Python and MySQL.
- Returns the database connection object whenever required.

3. dao Folder (Data Access Object)

admission_dao.py

This file contains all database-related operations.

It performs operations such as:

- Add Student
- Search Student
- Update Student
- Delete Student
- View All Students
- Get Course Details
- Get Course ID

The DAO layer separates SQL queries from the user interface, making the project more organized.

4. model Folder

student.py

- Represents the Student entity.
- Stores all student information using attributes.
- Provides getter and setter methods to access student data.
- Transfers student data between the GUI and the database.

5. controllers Folder

Admission_controller.py

- Acts as an intermediary between the View and the Service layer.
- Receives user requests from the graphical user interface.
- Forwards requests to the appropriate service methods.
- Coordinates the flow of data between the View and the Service layer.
- Keeps the presentation layer independent of the business logic.

6. services Folder

admission_service.py

- Contains the business logic of the application.
- Processes requests received from the Controller.
- Communicates with the DAO layer to perform database operations.
- Provides a separate layer for implementing future validations and business rules.
- Improves the maintainability, scalability, and reusability of the application.

7. view Folder

admission_form.py

- Creates the Student Admission Form using Tkinter.
- Accepts student information from the user.
- Provides buttons for Save, Search, Update, Delete, and Clear operations.

student_table.py

- Displays all student records in a table.
- Loads data from the MySQL database.
- Refreshes the table whenever records are added, updated, or deleted.

DATABASE DESIGN

The **College Admission Management System** uses **MySQL** as its database management system. The database is used to store all the information related to students and courses. It provides a secure and organized way to manage data and allows users to perform operations such as adding, searching, updating, and deleting records.

The database consists of different tables, each storing a specific type of information. These tables are connected using relationships, which help maintain data accuracy and avoid duplication.

In this project, the database stores details such as student information, course information, admission details, and fee records. Whenever a user operates the application, the data is automatically stored or updated in the MySQL database.

Using MySQL provides several advantages, such as fast data retrieval, secure storage, easy data management, and support for large amounts of information.

Benefits of Using MySQL

- Stores data securely.
- Provides fast searching and retrieval of records.
- Reduces data redundancy.
- Maintains data consistency.
- Supports easy updating and deletion of records.
- Handles a large amount of data efficiently.

DATABASE TABLES

The **College Admission Management System** uses a MySQL database to store student and course information. The database contains two main tables: **student** and **course**. These tables are connected using the **course_id** field.

Student Table

Field Name	Data Type	Description
student_id	INT	Unique ID of the student
student_name	VARCHAR(100)	Name of the student
father_name	VARCHAR(100)	Father's name
gender	VARCHAR(20)	Gender of the student
dob	DATE	Date of Birth
mobile	VARCHAR(15)	Mobile number
email	VARCHAR(100)	Email address
address	VARCHAR(100)	Residential address
admission_date	DATE	Date of admission
course_id	INT	Selected Course (Foreign Key)
fee_paid	DECIMAL(10,2)	Fee paid by the student
pending_fee	DECIMAL(10,2)	Remaining Fee
fee_status	VARCHAR	Fee Status (Paid/Pending)

Course Table

Field Name	Data Types	Description
course_id	INT	Unique Course ID (Primary Key)
course_name	VARCHAR	Name of the course
duration	VARCHAR	Duration of the course
course_fee	DECIMAL	Total fee of the course

Relationship Between Tables

The **student** table is connected with the **course** table through the **course_id** field. This relationship allows each student to be assigned to a specific course without storing the course details repeatedly in the student table.

IMPORTANT CODE SNIPPETS

1. MySQL Database Connection

```
class DBConnection:

    def __init__(self):
        self.connection = mysql.connector.connect(
            host="localhost",
            user="root",
            password="*****",
            database="college_admission_db"
        )

    def get_connection(self):
        return self.connection
```

Description

This code establishes the connection between the Python application and the MySQL database. It allows the application to perform database operations such as storing, retrieving, updating, and deleting student records.

2. Student Insertion (DAO Layer)

```
def add_student(self, student):

    query = """
    INSERT INTO student
    (
        student_name,
        father_name,
        gender,
        dob,
        mobile,
        email,
        address,
        admission_date,
        course_id,
        fee_paid,
        pending_fee,
        fee_status
    )
    VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
    """

    self.cursor.execute(query, values)
```

```
self.connection.commit()
```

Description

This method inserts a new student record into the MySQL database. It uses a parameterized SQL query to securely save the student's information.

3. Search Student

```
def search_student(self, student_id):

    query = """
    SELECT
    s.student_id,
    s.student_name,
    s.father_name,
    s.gender,
    s.dob,
    s.mobile,
    s.email,
    s.address,
    s.admission_date,
    s.course_id,
    c.course_name,
    c.course_fee,
    s.fee_paid,
    s.pending_fee,
    s.fee_status
    FROM student s
    INNER JOIN course c
        ON s.course_id = c.course_id
    WHERE s.student_id = %s
    """
    self.cursor.execute(query, (student_id,))

    return self.cursor.fetchone()
```

Description

This method searches for a student using the Student ID and returns the matching record from the database.

4. Update Student

```
def update_student(self, student):

    query = """
    UPDATE student
    SET
        student_name=%s,
        father_name=%s,
```

```

        gender=%s,
        dob=%s,
        mobile=%s,
        email=%s,
        address=%s,

        admission_date=%s,
        course_id=%s,
        fee_paid=%s,
        pending_fee=%s,
        fee_status=%s

WHERE student_id=%s
"""
Values=(
    student.get_student_name(),
    student.get_father_name(),
    student.get_gender(),
    student.get_dob(),
    student.get_mobile(),
    student.get_email(),
    student.get_address(),
    student.get_admission_date(),
    student.get_course_id(),
    student.get_fee_paid(),
    student.get_pending_fee(),
    student.get_fee_status(),
    student.get_student_id(),
)

self.cursor.execute(query, values)
self.connection.commit()

```

Description

This method updates an existing student's information in the database based on the Student ID.

5. Delete Student

```

def delete_student(self, student_id):

    query = """
DELETE FROM student
WHERE student_id=%s
"""

    self.cursor.execute(query, (student_id,))
    self.connection.commit()

```

Description

This method deletes a student record from the database using the Student ID.

PROJECT IMPLEMENTATION

The **College Admission Management System** was implemented using **Python**, **Tkinter**, and **MySQL**. The application provides a graphical user interface (GUI) that allows users to perform different admission-related operations easily. All student records are stored in the MySQL database and are managed through the application.

ID	Student Name	Father Name	Gender	DOB	Mobile	Email	Course	Paid Fee	Pending Fee
14	Berlin	Irving	Male	1954-09-04	6578945678	berlin@gmail.com	BCA	60000.00	0.00
15	Tokyo	Moscow	Female	1998-12-09	4535678976	tokyo@gmail.com	BCA	60000.00	0.00
18	Kanika	Shiv Kumar	Female	2006-09-08	8765678976	kanika@gmail.com	BCA	30000.00	30000.00
19	Albus Potter	Harry Potter	Male	1998-03-17	5678987656	albuspotter@gmail.com	BCA	40000.00	20000.00
20	Rose	Lotus	Male	2003-09-02	6758987654	rose@gmail.com	MCA	80000.00	0.00
21	Sheetal	Ramesh	Female	2006-07-28	2345678987	sheetalkumari@gmail.com	BBA	20000.00	50000.00
22	Junior White	Whitter White	Male	1998-12-08	9876543456	juniorwhite@gmail.com	MCA	34000.00	46000.00

Figure 1.1: Main Window of the College Admission Management System

The above figure shows the main interface of the **College Admission Management System** developed during the Summer Internship. The application is built using **Python**, **Tkinter**, and **MySQL**.

The upper section of the window contains the **Student Admission Form**, where users can enter student details such as Student Name, Father's Name, Gender, Date of Birth, Mobile Number, Email, Address, Course, Admission Date, Fee Paid, Pending Fee, and Fee Status.

The application provides the following operations through dedicated buttons:

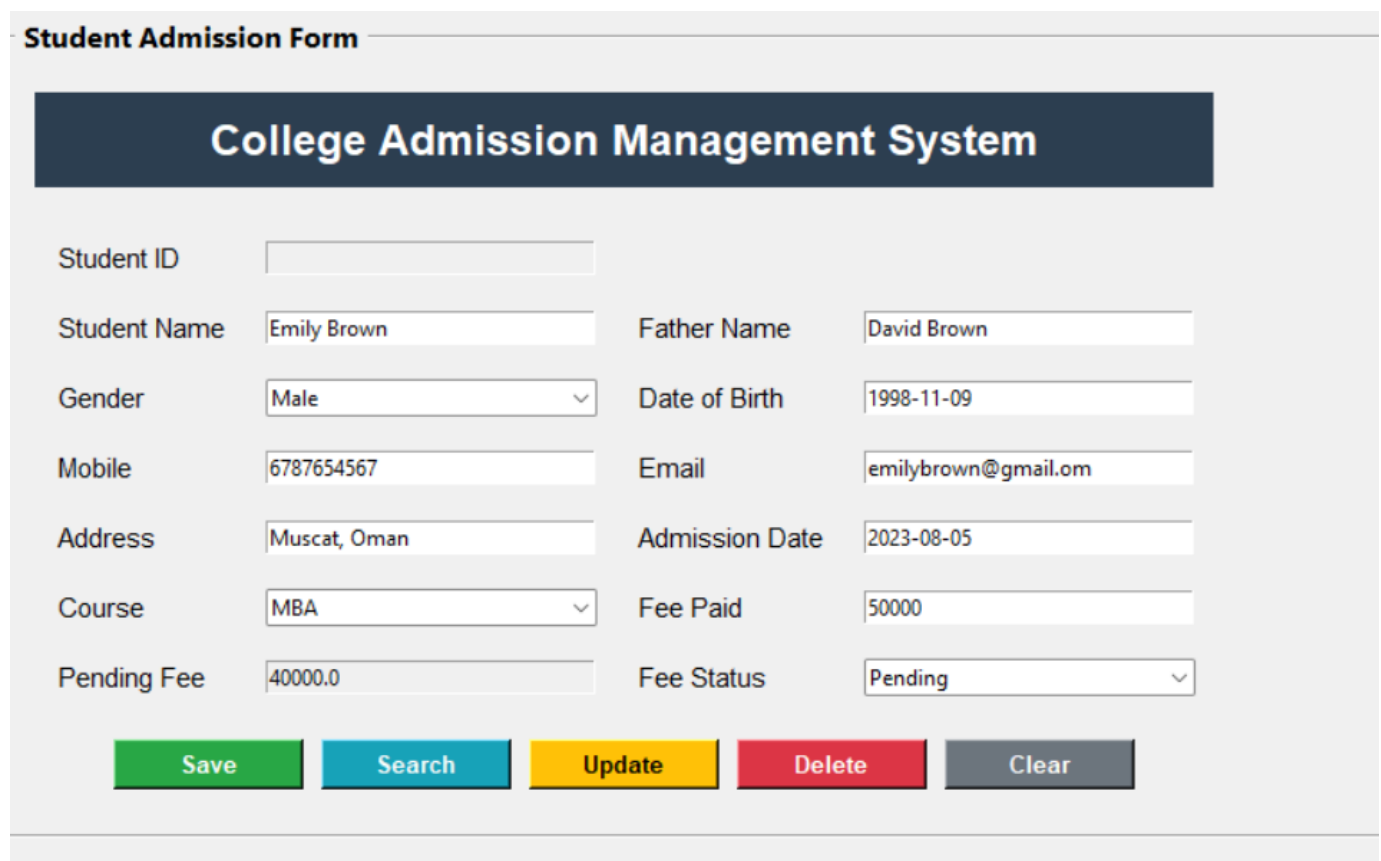
- **Save** – Stores a new student record in the database.
- **Search** – Finds a student using the Student ID.

- **Update** – Updates the details of an existing student.
- **Delete** – Deletes a student record from the database.
- **Clear** – Clears all the input fields.

The lower section displays the **Student Records Table**, which shows all the stored student records. The table allows users to view the admission details in an organized format.

This interface combines data entry and record management in a single window, making the application simple and easy to use.

Project Implementation – Student Admission Process



Student Admission Form

College Admission Management System

Student ID			
Student Name	Emily Brown	Father Name	David Brown
Gender	Male	Date of Birth	1998-11-09
Mobile	6787654567	Email	emilybrown@gmail.com
Address	Muscat, Oman	Admission Date	2023-08-05
Course	MBA	Fee Paid	50000
Pending Fee	40000.0	Fee Status	Pending

Figure 1.2: Student Admission Form

The Student Admission Form is one of the main modules of the **College Admission Management System**. It is used to register new students and collect all necessary admission-related information.

The form provides different input fields to enter student details such as Student Name, Father's Name, Gender, Date of Birth, Mobile Number, Email Address, Address, Admission Date, Course, Fee Paid, Pending Fee, and Fee Status.

The course selection is provided through a dropdown menu, which allows users to select available courses easily and reduces the possibility of incorrect data entry.

After entering all required information, the user can save the record by clicking the **Save** button. The application processes the entered data and sends it to the DAO layer, which performs the database operation and stores the information in the MySQL database.

Once the student record is successfully saved, the newly added information becomes available in the Student Records Table. This feature helps in maintaining student admission details digitally and reduces the dependency on manual record keeping.

Result

- Student details are entered successfully.
- User input is processed through the application.
- Student information is stored in the MySQL database.
- The admission record becomes available for future operations.

Project Implementation – Saving Student Record

The screenshot displays the 'Student Admission Form' interface within the 'College Admission Management System'. The form contains the following fields and values:

Field	Value
Student ID	
Student Name	Emily Brown
Father Name	David Brown
Gender	Male
Date of Birth	1998-11-09
Mobile	6787654567
Email	emilybrown@gmail.om
Address	Muscat, Oman
Admission Date	2023-08-05
Course	MBA
Fee Paid	50000
Pending Fee	40000.0
Fee Status	Pending

At the bottom of the form are five buttons: **Save** (green), **Search** (blue), **Update** (yellow), **Delete** (red), and **Clear** (grey). A 'Success' dialog box is open in the bottom right corner, displaying an information icon and the message 'Student Added Successfully.' with an 'OK' button.

Figure 1.3: Saving Student Record

The Save operation is used to store a new student's admission details permanently in the database. After entering all required information in the Student Admission Form, the user clicks the **Save** button to submit the record.

When the save operation is performed, the application collects the entered data from the GUI fields and sends it to the DAO layer. The DAO layer executes the SQL INSERT query and stores the student information in the MySQL database.

After successful insertion, the application confirms that the record has been saved and updates the Student Records Table to display the newly added student information.

This functionality helps maintain accurate digital records of students and eliminates the need for maintaining manual admission registers.

Result

- Student data is successfully inserted into the database.
- A new admission record is created.
- Student information is displayed in the records table.
- Data remains available for future operations like search, update, and delete.

Project Implementation – Viewing Student Record

The screenshot displays the 'College Admission Management System' interface. At the top, there is a 'Student Admission Form' with various input fields for student details. Below the form is a 'Student Records' table listing existing students. The table has columns for ID, Student Name, Father Name, Gender, DOB, Mobile, Email, Course, Paid Fee, and Pending Fee. The row for ID 24, Emily Brown, is highlighted with an orange border.

Student Admission Form

College Admission Management System

Student ID:

Student Name: Father Name:

Gender: Date of Birth:

Mobile: Email:

Address: Admission Date:

Course: Fee Paid:

Pending Fee: Fee Status:

Student Records

ID	Student Name	Father Name	Gender	DOB	Mobile	Email	Course	Paid Fee	Pending Fee
14	Berlin	Ining	Male	1954-09-04	6578945678	berlin@gmail.com	BCA	60000.00	0.00
15	Tokyo	Moscow	Female	1998-12-09	4535678976	tokyo@gmail.com	BCA	60000.00	0.00
18	Kanika	Shiv Kumar	Female	2006-09-08	8765678976	kanika@gmail.com	BCA	30000.00	30000.00
19	Albus Potter	Harry Potter	Male	1998-03-17	5678987656	albuspotter@gmail.com	BCA	40000.00	20000.00
20	Rose	Lotus	Male	2003-09-02	6758987654	rose@gmail.com	MCA	80000.00	0.00
21	Sheetal	Ramesh	Female	2006-07-28	2345678987	sheetalkumari@gmail.com	BBA	20000.00	50000.00
22	Junior White	Whitter White	Male	1998-12-08	9876543456	juniorwhite@gmail.com	MCA	34000.00	46000.00
23	Emily Brown	David Brown	Male	1998-11-09	6787654567	emilybrown@gmail.com	MBA	50000.00	40000.00

Figure 1.4: Student Record Table

The Student Records Table is used to display all registered student information stored in the **College Admission Management System**. It provides a structured view of admission records and allows users to easily access student details.

Whenever the application starts or a new record is added, updated, or deleted, the table retrieves the latest information from the MySQL database through the DAO layer.

The data is displayed using the Tkinter **Treeview** widget, which organizes the records into rows and columns. Each row represents a student's admission record, while columns represent different details such as personal information, course details, and fee information.

This feature helps users quickly view and manage student records without directly accessing the database. It improves data organization and makes the admission management process more efficient.

Result

- All student records are retrieved from the MySQL database.
- Information is displayed in a structured table format.
- Users can easily view admission details.
- Updated records are reflected automatically in the table.

Project Implementation – Viewing Student Record

The screenshot displays the 'College Admission Management System' interface. At the top, there's a header bar with the system name. Below it, the 'Student Admission Form' is visible, containing various input fields for student details. Below the form, there's a 'Student Records' table with columns for ID, Student Name, Father Name, Gender, DOB, Mobile, Email, Course, Paid Fee, and Pending Fee. The table lists several students, with the row for ID 22 (Junior White) highlighted in blue, indicating it is the selected record.

ID	Student Name	Father Name	Gender	DOB	Mobile	Email	Course	Paid Fee	Pending Fee
14	Berlin	Irving	Male	1954-09-04	6578945678	berlin@gmail.com	BCA	60000.00	0.00
15	Tokyo	Moscow	Female	1998-12-09	4535678976	tokyo@gmail.com	BCA	60000.00	0.00
18	Kanika	Shiv Kumar	Female	2006-09-08	8765678976	kanika@gmail.com	BCA	30000.00	30000.00
19	Albus Potter	Harry Potter	Male	1998-03-17	5678987656	albuspotter@gmail.com	BCA	40000.00	20000.00
20	Rose	Lotus	Male	2003-09-02	6758987654	rose@gmail.com	MCA	80000.00	0.00
21	Sheetal	Ramesh	Female	2006-07-28	2345678987	sheetalkumari@gmail.com	BBA	20000.00	50000.00
22	Junior White	Whitter White	Male	1998-12-08	9876543456	juniorwhite@gmail.com	MCA	34000.00	46000.00
23	Anna	Sunatel	Male	2000-10-23	7685645678	anna@gmail.com	MCA	45000.00	35000.00
24	Emily Brown	David Brown	Male	1998-11-09	6787654567	emilybrown@gmail.com	MBA	50000.00	40000.00

Figure 1.5: Search Student Record Operation

The Search Student feature allows users to retrieve complete information about an existing student by selecting a record from the **Student Records Table**.

When the user selects a particular student row from the table, the application identifies the selected student's ID and fetches the corresponding information from the MySQL database through the DAO layer.

The retrieved data, including student details, course information, and fee details, is automatically displayed in the admission form. This allows users to view the complete record and perform further operations such as updating or deleting the selected student's information.

This feature provides a convenient way to access student records quickly without manually searching through the database. It improves the usability of the system and makes student record management more efficient.

Result

- A student record is selected from the Student Records Table.
- The corresponding student information is retrieved successfully.
- Details are displayed automatically in the admission form.
- The selected record becomes available for further operations.

Project Implementation – Updating Student Record

The screenshot displays the 'College Admission Management System' interface. At the top, the 'Student Admission Form' is visible, containing fields for Student ID, Student Name, Father Name, Gender, Date of Birth, Mobile, Email, Address, Admission Date, Course, Fee Paid, Pending Fee, and Fee Status. Below the form are buttons for Save, Search, Update, Delete, and Clear. A 'Success' dialog box is overlaid on the form, stating 'Student Updated Successfully.' with an 'OK' button. Below the form is the 'Student Records' table, which lists student details including ID, Name, Father Name, Gender, DOB, Mobile, Email, Course, Paid Fee, and Pending Fee. The table has 10 columns and 10 rows of data.

ID	Student Name	Father Name	Gender	DOB	Mobile	Email	Course	Paid Fee	Pending Fee
14	Berlin	Irving	Male	1954-09-04	6578945678	berlin@gmail.com	BCA	60000.00	0.00
15	Tokyo	Moscow	Female	1998-12-09	4535678976	tokyo@gmail.com	BCA	60000.00	0.00
19	Albus Potter	Harry Potter	Male	1998-03-17	5678987656	albuspotter@gmail.com	BCA	40000.00	20000.00
20	Rose	Lotus	Male	2003-09-02	6758987654	rose@gmail.com	MCA	80000.00	0.00
21	Sheetal	Ramesh	Female	2006-07-28	2345678987	sheetalkumari@gmail.com	BBA	20000.00	50000.00
22	Junior White	Whitter White	Male	1998-12-08	9876543456	juniorwhite@gmail.com	MCA	34000.00	46000.00
23	Anna	Sunatol	Male	2000-10-23	7685643678	anna@gmail.com	MCA	45000.00	35000.00
24	Emily Brown	David Brown	Male	1998-11-09	6787654367	emilybrown@gmail.com	MBA	50000.00	40000.00

Figure 1.6: Update Student Record Operation

The Update Student feature allows users to modify the information of an existing student in the **College Admission Management System**. To update a record, the user first selects a student from the **Student Records Table**. The selected student's details are automatically loaded into the Student Admission Form.

The user can edit the required information, such as the student's personal details, contact information, course, or fee details. After making the necessary changes, the user clicks the **Update** button to save the modified information.

The application sends the updated student data to the DAO layer, which executes an SQL **UPDATE** query to modify the existing record in the MySQL database. Once the update operation is completed successfully, a confirmation message is displayed to the user, and the Student Records Table is refreshed automatically to show the latest information.

This feature ensures that student records remain accurate and up to date without creating duplicate entries in the database.

Result

- A student record is selected from the Student Records Table.
- Student information is modified successfully.

- Updated details are saved in the MySQL database.
- A success message confirms the completion of the update operation.
- The Student Records Table is refreshed with the latest information.

Project Implementation – Deleting Student Record

College Admission Management System

Student Admission Form

Student ID: 18
 Student Name: Kanika
 Father Name: Shiv Kumar
 Gender: Female
 Date of Birth: 2006-09-08
 Mobile: 8765678976
 Email: kanika@gmail.com
 Address: Rewari
 Admission Date: 2024-09-05
 Course: BCA
 Fee Paid: 30000.00
 Pending Fee: 30000.00
 Fee Status: Pending

Confirm Delete
 Are you sure you want to delete this student?
 Yes No

Student Records

ID	Student Name	Father Name	Gender	DOB	Mobile	Email	Course	Paid Fee	Pending Fee
14	Berlin	Iving	Male	1954-09-04	6578945678	berlin@gmail.com	BCA	60000.00	0.00
15	Tokyo	Moscow	Female	1998-12-09	4535678976	tokyo@gmail.com	BCA	60000.00	0.00
18	Kanika	Shiv Kumar	Female	2006-09-08	8765678976	kanika@gmail.com	BCA	30000.00	30000.00
19	Albus Potter	Harry Potter	Male	1998-03-17	5678987656	albuspotter@gmail.com	BCA	40000.00	20000.00
20	Rose	Lotus	Male	2003-09-02	6758987654	rose@gmail.com	MCA	80000.00	0.00
21	Sheetal	Ramesh	Female	2006-07-28	2345678987	sheetalkumari@gmail.com	BBA	20000.00	50000.00
22	Junior White	Whitter White	Male	1998-12-08	9876543456	juniorwhite@gmail.com	MCA	34000.00	46000.00
23	Anna	Sunatol	Male	2000-10-23	7685645678	anna@gmail.com	MCA	45000.00	35000.00
24	Emily Brown	David Brown	Male	1998-11-09	6787654567	emilybrown@gmail.com	MBA	50000.00	40000.00

Figure 1.7: Delete Student Record Operation

The Delete Student feature allows users to remove an existing student record from the **College Admission Management System**. Before deleting a record, the user selects a student from the **Student Records Table**. Once a record is selected, the student's complete information is automatically displayed in the admission form.

To prevent accidental deletion, the application displays a **confirmation dialog** when the **Delete** button is clicked. The user can choose **Yes** to permanently delete the selected record or **No** to cancel the operation.

If the user confirms the deletion, the application sends the selected student's ID to the DAO layer, which executes the SQL **DELETE** query in the MySQL database. After the record is removed successfully, the Student Records Table is refreshed automatically, ensuring that the deleted record is no longer displayed.

This confirmation mechanism improves data safety by preventing accidental deletion of student records and helps maintain the accuracy of the database.

Result

- A student record is selected from the Student Records Table.

- A confirmation dialog is displayed before deletion.
- The selected student record is deleted successfully from the MySQL database after confirmation.
- The Student Records Table is refreshed automatically to display the updated records.

Project Implementation – MySQL Database

The screenshot shows a MySQL database interface. At the top, there's a toolbar with icons for various database operations. Below the toolbar, a SQL query editor contains the following queries:

```

32 ('BBA', '3 Years', 70000),
33 ('B.Com', '3 Years', 50000),
34 ('MCA', '2 Years', 80000),
35 ('MBA', '2 Years', 90000);
36
37 • select * from course;
38 • desc student;
39 • desc course;
40 • select * from student;
41 • ALTER TABLE student

```

Below the queries, a 'Result Grid' displays a table of student records. The table has 13 columns: student_id, student_name, father_name, gender, dob, mobile, email, address, admission_date, course_id, fee_paid, fee_status, and pending_fee. The table contains 10 rows of data, including students like Berlin, Tokyo, Albus Potter, Rose, Sheetal, Junior White, Anna, and Emily Brown.

student_id	student_name	father_name	gender	dob	mobile	email	address	admission_date	course_id	fee_paid	fee_status	pending_fee
14	Berlin	Irving	Male	1954-09-04	6578945678	berlin@gmail.com	London	1975-10-07	1	60000.00	Paid	0.00
15	Tokyo	Moscow	Female	1998-12-09	4535678976	tokyo@gmail.com	Japan	2025-09-08	1	60000.00	Paid	0.00
19	Albus Potter	Harry Potter	Male	1998-03-17	5678987656	albuspotter@gmail.com	Hogwarts, London	2005-12-07	1	40000.00	Pending	20000.00
20	Rose	Lucus	Male	2003-09-02	6758987654	rose@gmail.com	Russia	2022-03-05	4	80000.00	Paid	0.00
21	Sheetal	Ramesh	Female	2006-07-28	2345678987	sheetalkumari@gmail.com	Sector-18, Rewari	2024-03-04	2	20000.00	Pending	50000.00
22	Junior White	Whitter White	Male	1998-12-08	9876543456	juniorwhite@gmail.com	Albuquerque, New Mexico	2022-03-04	4	34000.00	Pending	46000.00
23	Anna	Sunatol	Male	2000-10-23	7685645678	anna@gmail.com	Paris, France	2022-05-12	4	45000.00	Pending	35000.00
24	Emily Brown	David Brown	Male	1998-11-09	6787654567	emilybrown@gmail.com	Muscat, Oman	2023-08-05	5	50000.00	Pending	40000.00

Figure 1.8: Student Record Stored in MySQL Database

The **College Admission Management System** uses **MySQL** as the backend database to store and manage student admission records. All information entered through the Student Admission Form is stored permanently in the database.

Whenever a user performs operations such as **Save**, **Update**, **Search**, or **Delete**, the application communicates with the MySQL database through the DAO layer. SQL queries are executed to insert, retrieve, modify, or remove records as required.

The database stores important student information, including Student ID, Student Name, Father's Name, Gender, Date of Birth, Mobile Number, Email Address, Address, Admission Date, Course ID, Fee Paid, Pending Fee, and Fee Status.

Using MySQL ensures secure storage, fast data retrieval, and reliable management of admission records. It also maintains data consistency and supports efficient handling of student information.

Result

- Student records are stored securely in the MySQL database.
- Database operations are performed successfully using SQL queries.
- The application retrieves and updates records efficiently.
- Data remains consistent and available for future use.

SYSTEM TESTING

Software testing is an essential phase of software development that ensures the application performs all its functions correctly and meets the specified requirements. The **College Admission Management System** was tested after the completion of development to verify that all modules work properly and interact correctly with the MySQL database.

Each feature of the application, including student admission, searching, updating, deleting, and viewing records, was tested using different input values. The results confirmed that the application performs all operations successfully without any major errors.

Test Cases

Test Case	Input	Expected Result	Actual Result	Status
Add Student	Enter valid student details	Student record should be saved successfully	Record saved successfully	Passed
View Student Record	Open student record table	All student records should be displayed	Records displayed correctly	Passed
Search Student	Select a student record from the table	Student details should appear in the admission form	Student details displayed correctly	Passed
Update Student	Modify student information and click update	Updated information should be saved	Record updated successfully	Passed
Delete Student	Select a student and click Delete	Selected record should be removed after confirmation	Record deleted successfully	Passed
Database connection	Start the application	Application should be connect to MySQL	Connection established successfully	Passed

Testing Result

The testing results indicate that all major functionalities of the **College Admission Management System** work as expected. The application successfully performs CRUD (Create, Read, Update, Delete) operations, maintains data consistency, and communicates effectively with the MySQL database. The graphical user interface responds correctly to user actions, making the system reliable and user-friendly.

ADVANTAGES OF THE SYSTEM

Advantages of the system

The **College Admission Management System** provides several advantages over the traditional manual admission process. By automating admission-related tasks, the system improves efficiency, accuracy, and data management. The major advantages of the system are as follows:

Advantages

- Reduces manual paperwork by storing student information digitally.
- Saves time during the admission process by automating record management.
- Provides a user-friendly graphical interface that is easy to operate.
- Enables quick addition, searching, updating, and deletion of student records.
- Stores student information securely in the MySQL database.
- Reduces the chances of human errors and duplicate records.
- Displays all student records in a structured table for easy management.
- Automatically updates records after every database operation.
- Maintains fee information, including paid and pending fees, efficiently.
- Improves the overall organization and accessibility of admission data.
- Supports modular development using the MVC architecture and DAO design pattern, making the application easier to maintain and extend.

LIMITATIONS OF THE SYSTEM

Although the **College Admission Management System** provides an efficient solution for managing student admission records, it has certain limitations. These limitations can be addressed in future versions of the application to make the system more powerful and user-friendly.

Limitations

- The application is designed as a **desktop-based system** and can only be accessed on the computer where it is installed.
- The system supports **single-user access** and does not allow multiple users to work simultaneously.
- User authentication and role-based access control (such as Admin and Staff login) are not implemented.
- The application does not support **online admission** or remote access over the internet.
- Online fee payment and payment gateway integration are not available.
- Report generation in formats such as **PDF** or **Excel** is not included.
- Automatic data backup and recovery features are not implemented.
- Email or SMS notifications for admission confirmation are not available.
- The system is designed for admission management only and does not include modules such as attendance, examination, or library management.

LEARNING OUTCOMES

During the development of the **College Admission Management System**, I gained practical knowledge of software development concepts and improved my technical skills. The major learning outcomes are:

1. Understanding of MVC Architecture

I learned how to design an application using the **Model-View-Controller (MVC)** architecture. I understood the separation of database operations, user interface, and application logic, which helped in creating a structured and maintainable project.

2. Database Management Using MySQL

I gained hands-on experience with MySQL database integration. I learned how to create tables, establish relationships between tables, perform CRUD (Create, Read, Update, Delete) operations, and retrieve data using SQL queries.

3. Python GUI Development Using Tkinter

I improved my understanding of creating desktop applications using Tkinter. I learned how to design forms, handle user inputs, create tables using Treeview, and manage different GUI components.

4. Implementation of DAO Pattern

I learned how to use the Data Access Object (DAO) pattern to manage database operations separately from the user interface. This made the code more organized and easier to maintain.

5. Problem Solving and Debugging Skills

While developing the project, I faced various errors related to database connectivity, GUI updates, and data handling. Solving these issues improved my debugging and logical thinking skills.

6. Real-World Software Development Practices

I gained experience in planning features, testing functionalities, managing project structure, and developing a complete working application.

CHALLENGES FACED AND SOLUTIONS

During the project's development, I faced several challenges. The problems and solutions are mentioned below:

1. Database Connectivity Issues

Challenge:

Initially, connecting the Python application with the MySQL database was difficult due to configuration and connection errors.

Solution:

I carefully checked the database credentials, connection settings, and MySQL queries. After debugging the connection code, the application was successfully connected to the database.

2. Maintaining Data Synchronization Between GUI and Database

Challenge:

After adding or updating student records, the changes were not immediately reflected in the table displayed in the application.

Solution:

I implemented proper refresh functionality and reloaded the data from the database after every operation to keep the GUI synchronized with database records.

3. Implementing Update and Delete Operations

Challenge:

Handling update and delete operations while ensuring the correct student record was modified or removed was challenging.

Solution:

I used student IDs as unique identifiers and implemented SQL queries with proper conditions to update or delete only the selected record.

4. Managing Project Structure

Challenge:

Initially, managing multiple files such as GUI files, database connection files, and DAO classes was difficult.

Solution:

I organized the project using the MVC architecture and separated responsibilities into distinct modules, making it easier to understand and maintain.

5. Handling User Input Validation

Challenge:

Ensuring that users enter valid information in the admission form was challenging.

Solution:

I added validation checks for required fields and handled incorrect inputs to improve the reliability of the application.

CONCLUSION / SUMMARY

The internship proved to be a valuable learning experience that provided practical exposure to the software development process and helped me understand how theoretical concepts are applied in real-world applications.

During the internship, I successfully developed a **College Admission Management System** using **Python, Tkinter, and MySQL** by following a structured **MVC (Model-View-Controller) architecture** and **DAO (Data Access Object) pattern**. The system was designed to manage student admission records efficiently and included features such as student registration, course management, searching student records, updating and deleting records, fee management, and maintaining student information in a centralized database.

The project enhanced my understanding of Python GUI development, database connectivity, SQL operations, and software design practices. I gained hands-on experience in designing database tables, implementing CRUD operations, handling user inputs, and debugging different issues during the development process.

The challenges faced during the development improved my problem-solving and debugging skills. I also learned the importance of proper project planning, code organization, testing, and maintaining a clear separation between different components of an application.

The internship successfully connected my academic knowledge with practical software development. It provided me with confidence in developing database-driven applications and improved my technical skills in programming, database management, and application design. The knowledge and experience gained during this internship will serve as a strong foundation for my future career in software development.